MANUAL DEL ESTUDIANTE

Evaluación Automatizada de Calidad de Software

Guía paso a paso para subir tu proyecto a GitLab, pasar por el pipeline de SonarQube y obtener tu calificación oficial, sin sorpresas.

`git push` → `detect` → `quality` → `notify` → `nota / 20`

20.00

Nota máxima posible
100 % de la evaluación

CURSOS
Pruebas y Calidad de Software
Taller de Investigación I y II

PLATAFORMA
Quantum · GitLab + SonarQube

INSTITUCIÓN
Universidad Continental

CONTENIDO

Índice

Lee el manual en orden la primera vez. Después, usa la lista de verificación de la página final antes de cada entrega.

ANTES DE EMPEZAR

00 Cómo usar este manual y qué vas a lograr 3

01 Conceptos clave en 5 minutos 4

02 Cómo funciona el pipeline de calidad 5

PASO A PASO

P1 Acceso y registro en GitLab 6

P2 Generar tu token de acceso personal 7

P3 Preparar tu proyecto (los 2 archivos obligatorios + .gitignore) 8

P4 Crear el repositorio en GitLab 12

P5 Subir tu código (git push) 13

P6 Monitorear el pipeline 16

P7 Recibir y leer tu calificación 17

CÓMO SE CALIFICA

03 Reglas anti-trampas: lo que anula o resta tu nota 18

04 Rúbrica oficial de 0 a 20 puntos (con ejemplo resuelto) 19

05 Cobertura de pruebas unitarias por lenguaje 21

AYUDA

06 Solución de problemas frecuentes 22

07 Glosario 23

08 Lista de verificación antes de entregar 24

CAPÍTULO 00

Cómo usar este manual

Tu proyecto se califica de forma automática. No hay exposición ni revisión manual del código: lo que analiza la máquina es lo que vale.

Este manual explica **cada paso con el nivel de detalle de alguien que lo hace por primera vez**. Si ya dominas Git o GitLab, puedes avanzar rápido por esas partes, pero **no te saltes** los capítulos de reglas anti-trampas y de rúbrica: ahí están los errores que más puntos cuestan.

Lo que vas a lograr

1 · Tu código en GitLab
Una cuenta en la plataforma, un repositorio propio y tu proyecto subido con `git push`.

2 · Análisis automático
Un pipeline que revisa tu código con SonarQube en menos de 60 segundos.

3 · Nota oficial
Un correo a ti, a tu grupo y a tu docente con la nota sobre 20 y el detalle de cada problema.

Qué necesitas tener antes de empezar

Requisito	Detalle
Git instalado	Compruébalo abriendo una terminal y escribiendo `git --version`. Si responde con un número de versión, ya lo tienes. Si no, descárgalo desde git-scm.com.
Correo institucional	Con el formato `código@continental.edu.pe`. Ahí llega tu nota.
Tu proyecto	El código fuente de tu sistema (React, Vue, Angular, Node.js, Python, Java, Go, PHP, C#, etc.) en una carpeta de tu computadora.
Datos del grupo	Nombre completo, código y correo de cada integrante, y nombre y correo de tu docente evaluador.
Pruebas unitarias	Opcional para empezar, pero valen 4 de los 20 puntos (ver capítulo 05).

Cómo leer los recuadros

CONCEPTO
Explica una idea o término del paso.

CONSEJO
Atajo o buena práctica que ahorra tiempo.

ATENCIÓN
Error común que cuesta puntos o tiempo.

CRÍTICO
Puede anular tu nota o comprometer tu seguridad.

CAPÍTULO 01

Conceptos clave en 5 minutos

Estas seis piezas aparecen en todo el manual. Entenderlas hace que cada paso tenga sentido.

Git
Programa que guarda el historial de cambios de tu código en tu computadora. Cada "foto" del código se llama **commit**.

GitLab
Sitio web donde guardas una copia de tu repositorio en internet. Aquí vive tu entrega: `gitlab.quantumdevops.online`.

Repositorio (repo)
La carpeta de tu proyecto más su historial de Git. Hay una copia local (tu PC) y una remota (GitLab).

git push
El comando que envía tus commits locales al repositorio remoto. Cada push dispara el pipeline.

Pipeline CI/CD
Una cadena de tareas que GitLab ejecuta sola cada vez que subes código. Aquí, analiza tu calidad y te pone nota.

SonarQube
Herramienta que lee tu código sin ejecutarlo (análisis estático) y detecta bugs, vulnerabilidades, malas prácticas y código duplicado.

ANALOGÍA

Piensa en GitLab como la mesa de partes de la facultad. Haces `git push` (entregas tu trabajo), el pipeline es el evaluador que lo revisa con una rúbrica fija, y el correo es el acta de notas que llega a ti y a tu docente.

Los cuatro tipos de hallazgo que reporta SonarQube

Hallazgo	Qué significa	Ejemplo típico
Bug	Error de lógica que probablemente hace fallar el programa.	Comparar con `==` un valor que puede ser `null`.
Vulnerabilidad	Debilidad que un atacante podría aprovechar.	Construir una consulta SQL concatenando texto del usuario.
Code Smell	Código que funciona pero es difícil de mantener.	Funciones de 200 líneas, variables sin usar, nombres poco claros.
Security Hotspot	Código sensible que una persona debe revisar y marcar como seguro o no.	Uso de `Math.random()` para generar un token.

CAPÍTULO 02

Cómo funciona el pipeline

Es autónomo y desacoplado: tú solo agregas dos archivos y haces push. Todo lo demás lo hace la plataforma.

1 · Tu repositorio

`project.yml` con los datos académicos (curso, docente, integrantes) y `.gitlab-ci.yml`, que "enchufa" tu proyecto al pipeline oficial `root/laboratorio-quality`.

▼ git push

2 · Pipeline de calidad autónomo

Se ejecuta en tres etapas, una después de otra. Si una falla, las siguientes no corren.

ETAPA 1
detect
Valida `project.yml`, el curso y los correos del docente e integrantes.

ETAPA 2
quality
Corre SonarScanner, audita exclusiones (anti-trampas) y calcula la nota con la rúbrica.

ETAPA 3
notify
Envía el correo con la nota a ti, tu grupo y tu docente al mismo tiempo.

▼

3 · Resultados

Correo institucional con nota, desglose y retroalimentación por archivo y línea, más el panel de SonarQube en `sonar.quantumdevops.online`.

¿POR QUÉ ES TAN RÁPIDO?

El pipeline **no compila tu aplicación, no levanta contenedores ni crea bases de datos**. Solo lee tus archivos de código (análisis estático). Por eso termina en menos de 60 segundos.

CONSECUENCIA IMPORTANTE

Como el pipeline no ejecuta tu proyecto, **tampoco ejecuta tus pruebas**. El reporte de cobertura debe generarse en tu computadora y subirse junto con el código. Lo verás a detalle en el capítulo 05.

PASO 1

1 Acceso y registro en GitLab

Crea tu cuenta en la plataforma oficial. Solo se hace una vez por ciclo.

Si eres estudiante nuevo

1. Abre en tu navegador: **https://gitlab.quantumdevops.online**
2. Haz clic en la pestaña **Register now**.
3. Completa el formulario así:

Campo	Qué escribir	Ejemplo
Full name	Tus nombres y apellidos completos	Ana Lucía Torres Quispe
Username	Tu código de estudiante	`71234567`
Email	Tu correo institucional	`71234567@continental.edu.pe`
Password	Contraseña segura, mínimo 8 caracteres	Mezcla mayúsculas, números y símbolos

1. Confirma el registro y vuelve a iniciar sesión.

USA TU CÓDIGO COMO USUARIO

El *Username* forma parte de la dirección de tu repositorio (`gitlab.quantumdevops.online/71234567/...`). Usar tu código evita confusiones al identificar entregas.

Si olvidaste tu contraseña

1. En la pantalla de inicio de sesión, haz clic en **Forgot your password?**
2. Escribe el correo con el que te registraste.
3. En segundos recibirás un enlace de recuperación de **Plataforma Quantum** (`notificaciones@quantumdevops.online`). Revisa también la carpeta de spam.

ACCESO DESDE CUALQUIER RED

La plataforma está publicada en internet. Puedes usar la web, clonar y hacer `git push` desde tu casa, la universidad o tu celular con datos; no necesitas VPN.

PASO 2

2 Genera tu token de acceso

El token reemplaza a tu contraseña cuando Git se conecta a GitLab por HTTPS.

¿QUÉ ES UN PERSONAL ACCESS TOKEN?

Es una clave larga generada por GitLab (empieza con `glpat-`) que tiene **solo los permisos que tú elijas** y una fecha de caducidad. Si se filtra, puedes revocarla sin cambiar tu contraseña. Por eso es más seguro que usar tu contraseña real. Además, con HTTPS no necesitas generar llaves SSH ni abrir puertos, y en Windows y macOS Git lo recuerda tras el primer push.

2.1 · Entra a la sección de tokens

1. Inicia sesión en **https://gitlab.quantumdevops.online**
2. Haz clic en tu avatar (esquina superior derecha) → **Edit profile** (o **Preferences**).
3. En el menú lateral izquierdo, elige **Access Tokens**.

Atajo: abre directamente `https://gitlab.quantumdevops.online/-/user_settings/personal_access_tokens`

2.2 · Crea el token

1. Haz clic en **Add new token**.
2. Llena los campos:

Campo	Valor recomendado
Token name	`token-calidad` (o el nombre que prefieras)
Expiration date	La fecha por defecto, o el fin del ciclo académico
Select scopes	Marca **solo** estos dos: `read_repository` — permite leer y clonar tu repositorio `write_repository` — permite hacer `git push`

1. Haz clic en el botón azul **Create personal access token**.

2.3 · Copia y guarda el token

El token aparece en la parte superior de la página. **Cópialo de inmediato**: si recargas la página, no se volverá a mostrar y tendrás que crear otro.

TRÁTALO COMO UNA CONTRASEÑA

Guárdalo en un gestor de contraseñas, nunca dentro del proyecto, en `project.yml`, en un chat ni en una captura. Si crees que se filtró, ve a **Access Tokens**, pulsa **Revoke** y crea otro.

PASO 3

3 Prepara tu proyecto

Agrega dos archivos obligatorios en la raíz de tu proyecto. Te recomendamos un tercero que te protege de sacar 0.

¿QUÉ ES "LA RAÍZ" DEL PROYECTO?

Es la carpeta principal, la que contiene todo lo demás: donde normalmente están `package.json`, `pom.xml`, `requirements.txt` o `README.md`. Los archivos deben ir **ahí**, no dentro de `src/` ni de otra subcarpeta.

Estructura esperada

```
mi-proyecto/
├── .gitlab-ci.yml      ← obligatorio (activa el pipeline)
├── project.yml         ← obligatorio (datos académicos)
├── .gitignore          ← muy recomendado (evita la nota 0)
├── src/
├── tests/
└── package.json / pom.xml / requirements.txt ...
```

Archivo 1 · .gitlab-ci.yml

Le dice a GitLab que use el pipeline oficial de la universidad. Cópialo **exactamente** como está, respetando los espacios.

.gitlab-ci.yml

```yaml
# ============================================================
# ACTIVACIÓN DEL PIPELINE DE EVALUACIÓN DE CALIDAD (SONARQUBE)
# ============================================================
include:
  - project: "root/laboratorio-quality"
    file: "/gitlab/quality-pipeline.yml"
```

EL NOMBRE EMPIEZA CON PUNTO

El archivo se llama `.gitlab-ci.yml` (con punto inicial y extensión `.yml`). En Windows, verifica que no se haya guardado como `.gitlab-ci.yml.txt`: activa "Ver → Extensiones de nombre de archivo" en el Explorador.

Archivo 2 · project.yml

Contiene los datos de tu proyecto, tu curso, tu docente y tu grupo. El pipeline los usa para saber **a quién enviar la nota**.

project.yml

```yaml
project:
  name: "Sistema de Farmacia"          # Nombre comercial de tu sistema
  version: 1.0.0
  description: "Sistema web de gestión de inventarios y recetas"

stack:
  detect: true

# ============================================================
# METADATOS ACADÉMICOS PARA LA EVALUACIÓN
# ============================================================
academic:
  course: "Pruebas y Calidad de Software"
  evaluation_name: "Evaluación de Calidad"
  partial_weight_percentage: 100

  # Docente evaluador (recibe el correo oficial con la nota)
  evaluator:
    name: "Mg. Juan Pérez Docente"
    email: "docente@continental.edu.pe"
    role: "Docente Evaluador"

  # Integrantes del grupo (todos reciben copia de la nota)
  members:
    - name: "Ana Torres"
      code: "71234567"
      email: "71234567@continental.edu.pe"
    - name: "Luis Mendoza"
      code: "72345678"
      email: "72345678@continental.edu.pe"
```

Qué significa cada campo

Campo	Qué poner
`project.name`	El nombre de tu sistema. Aparece en el asunto del correo de la nota.
`project.version` / `description`	Versión y una frase que describa tu sistema.
`stack.detect`	Déjalo en `true` : el pipeline detecta solo tu lenguaje y framework.

`academic.course`	Exactamente uno de estos tres textos:
	`"Pruebas y Calidad de Software"`
	`"Taller de Investigación I"`
	`"Taller de Investigación II"`

`partial_weight_percentage`	Puedes poner 80 o 100. Hoy el motor siempre califica sobre el 100 % (0 a 20) sin aplicar descuentos por este valor.
`evaluator`	Nombre y correo **reales** de tu docente. Si el correo está mal, tu docente no recibe la nota.
`members`	Un bloque por integrante, cada uno con `name` , `code` y `email` .

YAML ES SENSIBLE A LOS ESPACIOS

- Usa **espacios, nunca tabulaciones**. Cada nivel se sangra con 2 espacios.
- El guion `-` de cada integrante debe quedar alineado con los demás.
- Pon los textos entre comillas, sobre todo los códigos (`"71234567"`), para que no se interpreten como números.
- Un error de sangría hace fallar la etapa `detect` y no recibirás nota. Puedes validar el archivo pegándolo en cualquier validador YAML en línea antes de subirlo.

Archivo 3 (recomendado) · .gitignore

Le dice a Git qué carpetas **no** subir. Sin él, puedes subir miles de archivos de librerías externas, SonarQube los analizará como si fueran tuyos y **tu nota se anulará a 0** (regla anti-saturación, capítulo 03).

.gitignore · Node / React / Vue / Angular

```
node_modules/
dist/
build/
.next/
.env
*.log
```

.gitignore · Java (Maven/Gradle)

```
target/
build/
.gradle/
*.class
.idea/
```

.gitignore · Python

```
venv/
.venv/
__pycache__/
*.pyc
.env
```

.gitignore · PHP / .NET

```
vendor/
bin/
obj/
.vs/
.env
```

NO IGNORES TU REPORTE DE COBERTURA

Muchas plantillas de `.gitignore` excluyen `coverage/` o `coverage.xml`. Como el pipeline no ejecuta tus pruebas, **si ignoras ese archivo SonarQube no lo verá y perderás hasta 4 puntos**. Revisa tu `.gitignore` y quita esas líneas si existen. Confirma con tu docente cualquier duda sobre cómo se recoge la cobertura.

PASO 4

4 Crea el repositorio en GitLab

El repositorio remoto es el “destino” al que enviarás tu código.

1. Inicia sesión en **https://gitlab.quantumdevops.online**
2. Haz clic en el botón `+` de la barra superior → **New project/repository**.
3. Elige **Create blank project**.
4. Completa los datos:

Campo	Qué elegir
Project name	Un nombre en minúsculas y con guiones, por ejemplo `calidad-sistema-farmacia`
Project URL	Selecciona tu usuario personal (tu código)
Visibility Level	`Internal` o `Public`
Initialize repository with a README	**DESMÁRCALA**

1. Presiona **Create project**. Copia la URL HTTPS que te muestra GitLab; la usarás en el siguiente paso.

¿POR QUÉ DESMARCAR EL README?

Si GitLab crea un README, el repositorio remoto ya tiene un commit que tu computadora no conoce. Al hacer `git push` recibirás un error de “historias no relacionadas” (*rejected / fetch first*). Con el repositorio vacío, tu primer push entra sin conflictos.

UN REPOSITORIO POR GRUPO

Basta con que **un integrante** cree el repositorio y haga el push. Todos los integrantes listados en `project.yml` reciben la nota por correo.

PASO 5

5 Sube tu código con git push

Ejecuta estos comandos en orden, dentro de la carpeta raíz de tu proyecto.

5.1 · Abre la terminal en la carpeta correcta

Sistema	Cómo hacerlo
Windows	Abre la carpeta del proyecto en el Explorador, clic derecho → **Open Git Bash here**. O en VS Code: menú **Terminal → New Terminal**.
macOS / Linux	Abre Terminal y escribe `cd` seguido de la ruta, por ejemplo `cd ~/Proyectos/mi-proyecto`.

Para confirmar que estás en el lugar correcto, escribe `ls` (o `dir` en PowerShell): debes ver `project.yml` y `.gitlab-ci.yml` en la lista (usa `ls -a` para ver archivos que empiezan con punto).

5.2 · Ejecuta los comandos

Terminal

```
# 1. Inicializar Git (solo si tu proyecto aún no es un repositorio)
git init -b main

# 2. Decir a Git quién eres (aparece como autor de los commits)
git config user.name "Tu Nombre Completo"
git config user.email "tu_correo@continental.edu.pe"

# 3. Preparar todos los archivos y registrar el commit
git add .
git commit -m "feat: entrega de evaluacion con pipeline de calidad"

# 4. Conectar tu carpeta con el repositorio de GitLab
git remote add origin https://gitlab.quantumdevops.online/TU_USUARIO/TU_PROYECTO.git

# 5. Enviar el código al servidor
git push -u origin main
```

Comando	Qué hace
`git init -b main`	Convierte la carpeta en un repositorio Git con la rama principal llamada `main`.
`git config user.*`	Registra tu nombre y correo como autor de los cambios en este proyecto.
`git add .`	Marca todos los archivos (menos los de `.gitignore`) para el próximo commit. El punto significa "todo en esta carpeta".
`git commit -m "..."`	Toma la "foto" del código con un mensaje que describe el cambio.
`git remote add origin URL`	Guarda la dirección de tu repositorio de GitLab con el apodo `origin`.
`git push -u origin main`	Sube la rama `main` a `origin`. El `-u` la deja vinculada para que después baste con `git push`.

ANTES DEL COMMIT, REVISA QUÉ VAS A SUBIR

Ejecuta `git status` después de `git add .`. Si ves `node_modules/`, `venv/`, `vendor/` o `target/` en la lista, **detente**: falta o está mal tu `.gitignore`.

5.3 · Autenticación

Al ejecutar `git push`, Git te pedirá credenciales (a veces en una ventana emergente):

Te pide	Escribe
Username	Tu usuario de GitLab, por ejemplo `71234567`
Password	Pega tu **token** `glpat-...` del Paso 2 (no tu contraseña de GitLab). Al pegarlo no verás caracteres en pantalla; es normal.

En Windows y macOS, el gestor de credenciales de Git recordará el token y no te lo volverá a pedir.

5.4 · Atajo con el token en la URL (úsalo con cuidado)

Puedes evitar que Git te pida credenciales incluyendo el token en la dirección del remoto:

```
Terminal
git remote add origin https://TU_USUARIO:TU_TOKEN@gitlab.quantumdevops.online/TU_USUARIO/TU_PROYECTO.git
git push -u origin main
```

RIESGO DE SEGURIDAD

Con este método el token queda guardado **en texto plano** dentro de `.git/config` y en el historial de tu terminal. Si compartes la carpeta comprimida o alguien usa tu PC, puede copiarlo. Úsalo solo en tu computadora personal; si ya lo usaste en una PC compartida, revoca el token. Para cambiar a la URL sin token: `git remote set-url origin https://gitlab.quantumdevops.online/TU_USUARIO/TU_PROYECTO.git`

5.5 · Entregas posteriores

Cada vez que mejores tu código y quieras una nueva calificación, basta con:

Terminal

```
git add .
git commit -m "fix: corrige vulnerabilidades reportadas por SonarQube"
git push
```

Cada push genera un nuevo pipeline y un nuevo correo. Pregunta a tu docente cuál entrega toma como oficial (por ejemplo, la última antes de la fecha límite).

PASO 6

6 Monitorea el pipeline

Comprueba en GitLab que las tres etapas terminen en verde.

1. Entra a tu proyecto en GitLab.
2. En el menú lateral: **Build** → **Pipelines**.
3. Haz clic en el pipeline más reciente para ver sus tres etapas.

Etapa	Qué hace	Si falla, revisa...
`detect`	Verifica que exista `project.yml` y lee el curso, la configuración académica y los correos.	Que `project.yml` esté en la raíz, con sangría correcta y correos válidos.
`quality`	Ejecuta SonarScanner, audita exclusiones (anti-trampas) y calcula la nota sobre 20.	El log del job: busca mensajes de exclusión ilegítima o exceso de hallazgos.
`notify`	Empaqueta el reporte y envía los correos.	Que los correos de `project.yml` estén bien escritos.

Cómo interpretar los estados

Estado	Significado
passed	Todo terminó bien. Tu nota ya fue enviada. Ojo: *passed* no significa nota alta, solo que el proceso funcionó.
running	Está en curso. Espera; normalmente toma menos de 60 segundos.
pending	En cola, esperando un servidor libre. Puede pasar en horas de mucha demanda (cerca de la fecha límite).
failed	Algo falló. Haz clic en la etapa en rojo para ver el log y busca el error. Consulta el capítulo 06.

NO DEJES LA ENTREGA PARA EL ÚLTIMO MINUTO

Si muchos estudiantes hacen push a la vez, los pipelines pueden quedar en cola. Haz una entrega de prueba días antes para comprobar que todo funciona y luego mejora tu código con calma.

PASO 7

7 Recibe y lee tu calificación

Tu docente y todos los integrantes reciben el mismo correo al mismo tiempo.

El correo

De: Plataforma Quantum <notificaciones@quantumdevops.online>

Asunto: Calificación [Nombre de Proyecto] · Nota: XX.XX / 20 · Universidad Continental

El correo contiene:

- **Calificación oficial** del proyecto, por ejemplo `17.50 / 20.00`, directamente en escala vigesimal (100 % de la evaluación).
- **Desglose con medallas** por cada uno de los 6 criterios de la rúbrica.
- **Detalle de incidencias** por archivo y número de línea, con la solución recomendada.

¿NO LLEGÓ EL CORREO?

- Revisa spam, "Otros" o "Promociones".
- Confirma en GitLab que la etapa `notify` terminó en verde.
- Revisa que tu correo esté bien escrito en `project.yml`.

El panel de SonarQube

Para ver el análisis completo y las líneas exactas de código afectadas, entra a **https://sonar.quantumdevops.online**. Desde ahí puedes:

- Ver cada bug, vulnerabilidad y code smell con la explicación de la regla que incumple.
- Revisar los **Security Hotspots** y marcarlos como seguros o corregirlos (sube tu puntaje en el criterio 5).
- Identificar los bloques de código duplicado.

ESTRATEGIA PARA SUBIR TU NOTA

Ataca primero lo que más pesa: **vulnerabilidades (5 pts)**, luego **bugs, cobertura y code smells (4 pts cada uno)**. Corrige, haz commit y push, y compara el nuevo correo con el anterior.

CAPÍTULO 03

Reglas anti-trampas

El motor audita automáticamente tu configuración. Estas dos reglas pueden quitarte la mitad de la nota o anularla por completo.

REGLA 1 · EXCLUSIÓN ILEGÍTIMA DE CARPETAS: −10.0 PUNTOS

SonarQube permite excluir archivos del análisis mediante `sonar-project.properties`. La plataforma lo limita así:

- **Permitido:** excluir hasta **5 archivos individuales** de configuración, como `vite.config.ts` o `tailwind.config.js`.
- **Prohibido:** excluir **carpetas enteras de código fuente** como `src/**`, `app/*`, `backend/**`, `routes/**`.
- Si el auditor detecta lo segundo, lo cataloga como **fraude** y resta **10 puntos de la nota final**.

✔ Aceptable · sonar-project.properties

```
sonar.exclusions=vite.config.ts,tailwind.co
nfig.js
```

✘ Penalizado (−10)

```
sonar.exclusions=src/**,backend/**
```

No necesitas crear `sonar-project.properties` para que el pipeline funcione. Solo es relevante si decides excluir archivos.

REGLA 2 · ANTI-SATURACIÓN: MÁS DE 2500 HALLAZGOS = 0.00 / 20

- Si la suma **Bugs + Vulnerabilidades + Code Smells + Hotspots supera 2500**, el sistema asume que subiste dependencias de terceros sin filtrar (`node_modules`, `venv`, `vendor`, `.git`).
- En ese caso **la nota queda anulada en 0.00 / 20**, sin importar la calidad de tu código real.
- **Prevención:** crea un `.gitignore` adecuado *antes* del primer commit (ver Paso 3).

Si ya subiste node_modules por error

Agregar la carpeta al `.gitignore` después no basta: Git ya la está siguiendo. Debes quitarla del índice:

Terminal

```
# 1. Agrega la carpeta a .gitignore (si aún no está)
echo "node_modules/" >> .gitignore

# 2. Deja de seguirla en Git SIN borrarla de tu disco
git rm -r --cached node_modules

# 3. Registra y sube el cambio
git commit -m "chore: quita dependencias del repositorio"
git push
```

Cambia `node_modules` por `venv`, `vendor` o `target` según tu lenguaje.

CAPÍTULO 04

Rúbrica oficial (0 a 20)

Seis dimensiones con pesos fijos. La nota es la suma directa de los puntos de cada una.

Dimensión	Peso	Máx.	Cómo se asignan los puntos
1. Seguridad Vulnerabilidades	25 %	**5.00**	A 5.00 B 3.75 C 2.50 D 1.25 E 0.00
2. Fiabilidad Bugs	20 %	**4.00**	A 4.00 B 3.00 C 2.00 D 1.00 E 0.00
3. Cobertura de pruebas Pruebas unitarias	20 %	**4.00**	≥ 80 % → 4.00 65–80 % → 3.00 50–65 % → 2.00 30–50 % → 1.00 < 30 % → 0.00
4. Mantenibilidad Code Smells	20 %	**4.00**	A 4.00 B 3.00 C 2.00 D 1.00 E 0.00
5. Security Hotspots % revisados	10 %	**2.00**	≥ 80 % → 2.00 70–80 % → 1.50 50–70 % → 1.00 30–50 % → 0.50 < 30 % → 0.00 Sin hotspots → 2.00 automáticos
6. Duplicación % de código duplicado	5 %	**1.00**	≤ 3 % → 1.00 3–5 % → 0.75 5–10 % → 0.50 10–20 % → 0.25 > 20 % → 0.00
TOTAL	**100 %**	**20.00**	**Nota directa sobre 20 puntos**

¿QUÉ SON LAS LETRAS A–E?

SonarQube asigna una calificación por letra según la **severidad del peor problema** encontrado en esa dimensión. Por ejemplo, en seguridad basta **una sola vulnerabilidad grave** para bajar la letra de A a D o E. Por eso conviene corregir primero los hallazgos de mayor severidad (*Blocker* y *Critical*).

Ejemplo resuelto

Un grupo obtiene estos resultados en SonarQube:

Dimensión	Resultado del análisis	Puntos
Seguridad	Calificación A (0 vulnerabilidades)	5.00
Fiabilidad	Calificación B (1 bug menor)	3.00
Cobertura	72 % (rango 65–80 %)	3.00
Mantenibilidad	Calificación A	4.00
Security Hotspots	6 de 10 revisados = 60 % (rango 50–70 %)	1.00
Duplicación	2.4 % (≤ 3 %)	1.00
Nota final		**17.00**

¿Cómo subirla? Revisar 2 hotspots más (80 %) suma +1.00; corregir el bug suma +1.00; subir la cobertura a 80 % suma +1.00. Con eso llegaría a **20.00**.

Si el mismo grupo hubiera excluido `src/**`, la nota sería 17.00 − 10.00 = 7.00. Si hubiera subido `node_modules` y superado 2500 hallazgos, sería 0.00.

CAPÍTULO 05

Cobertura de pruebas por lenguaje

Para obtener los 4 puntos completos, SonarQube debe encontrar el informe de cobertura de tus pruebas.

¿QUÉ ES LA COBERTURA?

Porcentaje de líneas de tu código que se ejecutan al correr tus pruebas (800 de 1000 líneas = 80 %). La herramienta de pruebas genera un **reporte de cobertura** que SonarQube lee.

Flujo recomendado

1. Escribe tus pruebas unitarias, priorizando la lógica de negocio (servicios, validaciones, cálculos).
2. Ejecuta el comando de tu lenguaje (tabla siguiente) **en tu computadora**.
3. Verifica que se creó el archivo de reporte en la ruta indicada.
4. Asegúrate de que `.gitignore` **no** lo excluya.
5. Haz `git add`, `commit` y `push`.

Lenguaje / herramienta	Comando	Archivo que debe generarse
Node.js / TypeScript / React / Vue Jest o Vitest	`npm test -- --coverage`	`coverage/lcov.info`
Python pytest + pytest-cov	`pytest --cov=src --cov-report=xml:coverage.xml`	`coverage.xml`
Java Maven/Gradle + JaCoCo	`mvn clean test jacoco:report`	`target/site/jacoco/jacoco.xml`
Go	`go test -coverprofile=coverage.out ./...`	`coverage.out`
C# / .NET	`dotnet test --collect:"XPlat Code Coverage"`	`TestResults/**/coverage.cobertura.xml`

DETALLES POR LENGUAJE

- **Vitest** necesita el proveedor de cobertura instalado (`@vitest/coverage-v8`) y el reporter `lcov` activado en su configuración.
- **Python**: instala `pytest-cov` (`pip install pytest-cov`) y cambia `--cov=src` por el nombre real de tu carpeta de código.
- **Java**: el plugin de JaCoCo debe estar declarado en tu `pom.xml` o `build.gradle`.
- **Target/ y bin/**: en Java y .NET esas carpetas suelen estar en `.gitignore`; si el reporte vive ahí, agrega una excepción solo para el archivo de reporte (por ejemplo, `!target/site/jacoco/jacoco.xml`).

CAPÍTULO 06

Solución de problemas

Los errores más comunes, qué los causa y cómo resolverlos.

Síntoma	Causa probable	Solución
`git: command not found` o "git no se reconoce"	Git no está instalado o la terminal no lo encuentra.	Instala Git desde git-scm.com y abre una terminal nueva.
`Authentication failed` al hacer push	Usaste tu contraseña en lugar del token, o el token expiró o no tiene `write_repository`.	Crea un token nuevo con los dos permisos y pégalo como contraseña.
`remote origin already exists`	Ya habías configurado un remoto.	`git remote set-url origin URL_CORRECTA`
`rejected ... fetch first`	El repositorio de GitLab se creó con README.	`git pull origin main --allow-unrelated-histories`, resuelve conflictos y vuelve a hacer push.
`src refspec main does not match any`	Aún no hiciste ningún commit, o tu rama se llama `master`.	Haz el commit primero; revisa tu rama con `git branch`.
No aparece ningún pipeline	`.gitlab-ci.yml` no está en la raíz o tiene otro nombre.	Revisa el nombre exacto y la ubicación; vuelve a hacer push.
Etapa `detect` en rojo	`project.yml` ausente, mal sangrado o con datos inválidos.	Valida el YAML, usa espacios en vez de tabulaciones y revisa los correos.
Nota 0.00 / 20	Más de 2500 hallazgos por dependencias subidas.	Usa `.gitignore` y `git rm -r --cached` (capítulo 03).
Penalización de −10	Excluiste carpetas de código en `sonar-project.properties`.	Deja solo hasta 5 archivos individuales en `sonar.exclusions`.
Cobertura 0 % teniendo pruebas	El reporte no se generó, está en otra ruta o lo ignora `.gitignore`.	Revisa el capítulo 05 y verifica con `git status` que el reporte se suba.
No llega el correo	Correo mal escrito, filtro de spam o `notify` fallido.	Revisa spam, el estado de `notify` y los correos en `project.yml`.

CAPÍTULO 07

Glosario

Términos técnicos usados en este manual.

Análisis estático
Revisión del código leyendo los archivos, sin ejecutar el programa.

Branch (rama)
Línea de desarrollo independiente. La principal suele llamarse `main`.

CI/CD
Integración y entrega continuas: automatizar tareas cada vez que se sube código.

Code Smell
Señal de código difícil de mantener, aunque funcione.

Commit
Registro de un conjunto de cambios con un mensaje descriptivo.

Cobertura
Porcentaje del código que ejecutan las pruebas unitarias.

Dependencias
Librerías de terceros que usa tu proyecto (`node_modules`, `venv`...). No se suben al repositorio.

.gitignore
Archivo con la lista de carpetas y archivos que Git no debe subir.

Hallazgo (issue)
Cualquier problema detectado por SonarQube.

HTTPS
Protocolo web cifrado; aquí se usa para conectar Git con GitLab.

Job
Tarea individual dentro de una etapa del pipeline.

LCOV / Cobertura XML
Formatos de archivo de reporte de cobertura que SonarQube sabe leer.

origin
Apodo convencional del repositorio remoto en GitLab.

Pipeline
Secuencia automática de etapas que GitLab ejecuta tras cada push.

Push
Enviar tus commits locales al repositorio remoto.

Raíz del proyecto
Carpeta principal que contiene todo el proyecto.

Repositorio
Proyecto bajo control de versiones, con todo su historial.

Scope
Permiso concreto que se le otorga a un token.

Security Hotspot
Código sensible que requiere revisión humana para confirmar si es seguro.

SonarQube / SonarScanner
Plataforma de análisis de calidad y el programa que envía tu código a analizar.

Stage (etapa)
Fase del pipeline: aquí `detect`, `quality` y `notify`.

Token (PAT)
Clave de acceso con permisos limitados que reemplaza a tu contraseña en Git.

YAML
Formato de texto para configuración, basado en sangría con espacios.

CAPÍTULO 08

Lista de verificación final

Revísala antes de cada entrega. Si todas las casillas están marcadas, estás listo.

Cuenta y acceso

- ☐ Tengo cuenta en GitLab con mi código como usuario.
- ☐ Generé un token con `read_repository` y `write_repository`.
- ☐ Guardé el token en un lugar seguro, fuera del proyecto.

Archivos del proyecto

- ☐ `.gitlab-ci.yml` está en la raíz, con el contenido exacto.
- ☐ `project.yml` está en la raíz, sin tabulaciones.
- ☐ El curso coincide con una de las tres opciones válidas.
- ☐ Los correos del docente e integrantes son correctos.
- ☐ Existe un `.gitignore` adecuado a mi lenguaje.

Calidad y reglas

- ☐ `git status` no muestra `node_modules`, `venv`, `vendor` ni `target`.
- ☐ No excluyo carpetas de código en `sonar-project.properties`.
- ☐ Generé el reporte de cobertura y no está ignorado.
- ☐ Revisé los Security Hotspots en SonarQube.

Entrega

- ☐ El repositorio de GitLab se creó sin README.
- ☐ Hice `git push` a la rama `main`.
- ☐ El pipeline terminó en **passed**.
- ☐ Recibí el correo con la nota.

CÓMO PEDIR AYUDA DE FORMA EFECTIVA

Cuando consultes a tu docente, adjunta: el enlace a tu repositorio, el enlace al pipeline fallido, una captura del mensaje de error del log y lo que ya intentaste. Así la respuesta será mucho más rápida.

Enlaces oficiales

Recurso	Dirección
GitLab	https://gitlab.quantumdevops.online
Tokens de acceso	https://gitlab.quantumdevops.online/-/user_settings/personal_access_tokens
SonarQube	https://sonar.quantumdevops.online
Correo de la plataforma	notificaciones@quantumdevops.online